



SISTEMA DE CRAFTEO

Paradigmas de Programación – Primer cuatrimestre 2025



Docentes
Gasior, Federico Mauro
Lanzillotta, Hernán Gabriel

Integrantes - Grupo Iota

DNI	Apellido y nombre
CENSURED	Hoz, Lucas Nahuel
CENSURED	Maudet, Alejandro Nicolas

1 DE JULIO DE 2025

TRABAJO PRÁCTICO N° 2

Universidad Nacional de La Matanza (UNLaM)

Índice

Introducción	2
Desarrollo	2
Requisitos del sistema	2
Diagrama de clases	3
Aclaraciones.....	4
Conclusiones	4
Invitación	4
Referencias	4

Introducción

En el presente documento se aborda el diagrama de clases de una aplicación que simula un sistema de crafeo desarrollado en Java. Este diagrama se encuentra acompañado de una breve descripción de cada clase que lo compone, junto con sus responsabilidades e interrelaciones que permitieron confeccionarlo.

Desarrollo

Requisitos del sistema

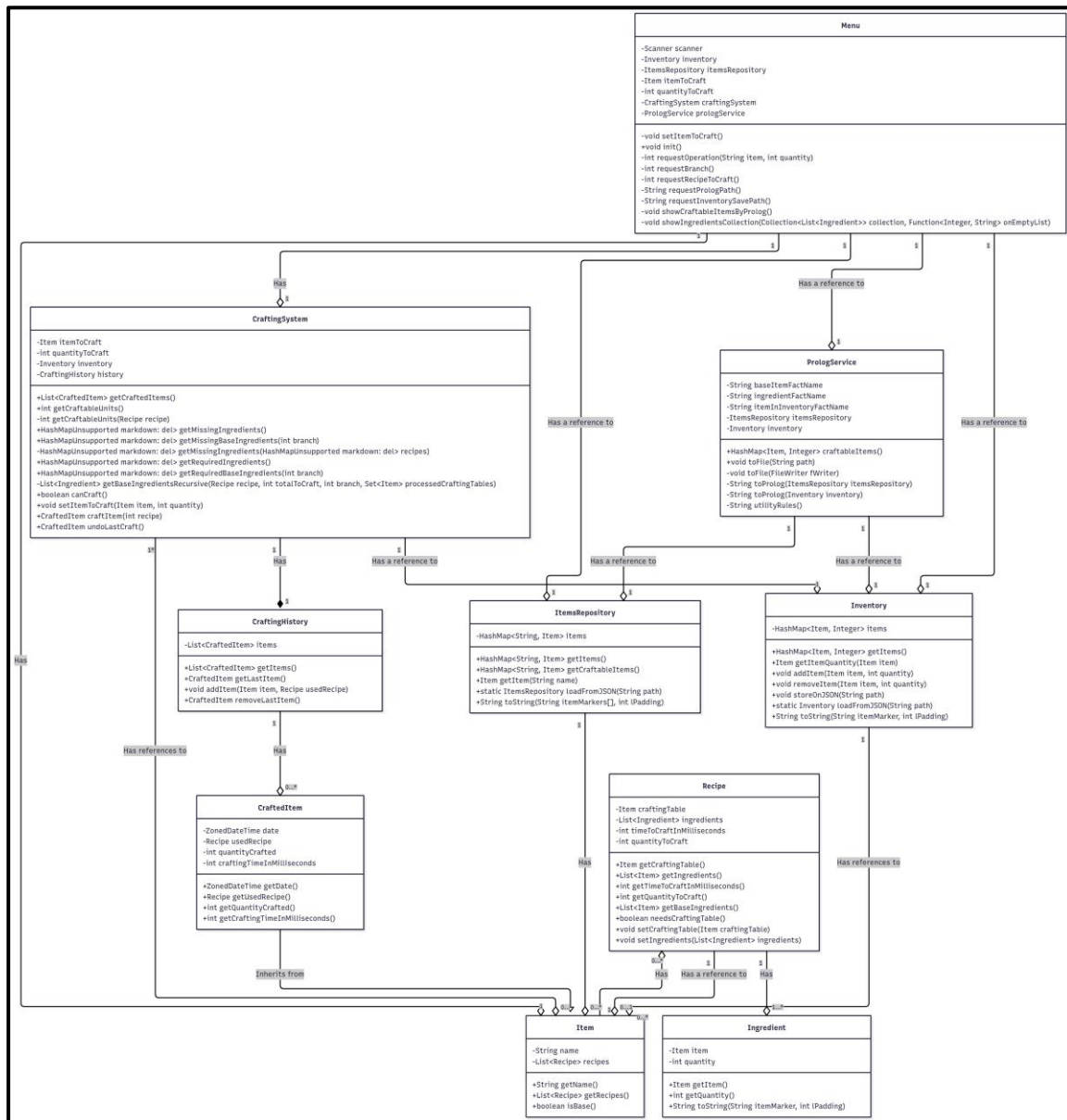
En los requisitos del sistema se nos presentan dos vertientes, los requisitos esenciales y los deseables. Los **requisitos esenciales** son aquellos que el sistema debe cumplir como mínimo para ser viable su puesta en producción, y estos son:

- Craftear un item
- Registrar los items crafeados en un historial
- Mostrar el tiempo necesario para craftear un item
- Mostrar la lista de ingredientes faltantes para craftear un item
- Pruebas unitarias de cada funcionalidad del sistema utilizando [JUnit 5](#)
- Mostrar la lista de ingredientes básicos faltantes para craftear un item
- Mostrar la lista de ingredientes y cantidades necesarias para craftear un item
- Mostrar la cantidad de items crafeables en base a los ingredientes en el inventario
- Mostrar la lista de ingredientes básicos y sus cantidades necesarias para craftear un item

Mientras que los requisitos deseables son aquellos que, en caso de disponer del tiempo necesario para desarrollarlos, suman puntos al producto (software) final, pero no son fundamentales para su correcto funcionamiento. Los **requisitos deseables** de este sistema son:

- Mostrar árbol de crafeos
- Craftear mesas de crafeo
- Guardar el inventario final en un archivo JSON
- Craftear items con más de una receta posible
- Craftear ciertos items únicamente si se posee la mesa de crafeo adecuada

Diagrama de clases



En el presente diagrama se pueden observar las clases que componen el sistema de crafeo. Como se puede notar gran parte de las clases poseen referencias a la clase **ItemsRepository**, siendo esta la fuente de la verdad que es consultada para obtener todos los items del dominio (items base y crafeables).

Aunque la finalidad de este documento no es ser un informe detallado del diagrama de clases, es importante mencionar las responsabilidades y funciones de las clases más fundamentales del sistema:

- **Menu:** clase encargada de proporcionar la interfaz gráfica para que el usuario final pueda interactuar con el sistema.
- **CraftingSystem:** administra los casos de uso del sistema, comunicándose con las diferentes clases del sistema para producir el resultado esperado.

- **PrologService:** servicio que se comunica con el sistema de [Prolog](#) para ejecutar consultas sobre hechos y reglas establecidas en base al inventario asociado y el repositorio de items.
- **ItemsRepository:** brinda la fuente de verdad para que el sistema conozca los items del dominio y las recetas asociadas a estos.
- **Inventory:** repositorio donde se almacenan los items que el usuario final tiene e irá adquiriendo según los crafteos y operaciones que realice en el ciclo de vida del sistema.

Aclaraciones

- I. Se han creado clases utilitarias para modificar cadenas de texto, excepciones personalizadas y clases constructoras (**Builders**) que no se han incluido en el diagrama de clases a fin de evitar una sobrecarga de información.
- II. Al ejecutar el método **craftableItems()** de la clase **PrologService** este no tiene en cuenta si los ingredientes faltantes pueden ser crafteados. Sin embargo, esto si se tiene en cuenta en el método **getCraftableUnits()** de la clase **CraftingSystem**.

Conclusiones

Observando el diagrama podemos concluir que el mismo se adapta a los requisitos del sistema, teniendo una buena cohesión y bajo acoplamiento. Gracias a estas características podemos no solo desarrollar un software que cumple con los requisitos establecidos, sino que también un software que es escalable en el tiempo y fácil de mantener.

Invitación

Si deseas conocer a detalle la arquitectura del sistema, visita el [repositorio de GitHub](#) dónde se encuentra alojado. Dentro de este podrás encontrar una versión detallada que incluye la guía de instalación, un diagrama interactivo, la organización del equipo de desarrollo, la licencia asociada, etc.

Referencias

[JUnit team](#). JUnit 5 User Guide. JUnit. <https://docs.junit.org/current/user-guide/>

[SWI Prolog](#). Manual. SWI-Prolog. https://www.swi-prolog.org/pldoc/doc_for?object=manual

[Object Management Group](#). Welcome to UML Web Site. UML ORG. <https://www.uml.org/>